

Agentic AI - Complete Course Notes

Video: Complete Agentic AI Course (10+ Hours)

Krishna - YouTube | youtube.com/watch?v=rV3HJ4LEZ7k

Beginner-friendly notes in simple language with examples. Topics: LangChain, LangGraph, RAG, Vectorless RAG, Deep Agents, Guardrails, LLM Evaluation, LLM Gateways + Top Interview Questions.

Video Timestamps (Course Map)

- 00:00:00 Introduction - GenAI vs Agentic AI overview
- 00:02:31 LangChain Course (~2.5 hrs) - models, tools, agents, middleware
- 02:35:12 LangGraph Course (~2.5 hrs) - StateGraph, nodes, edges, agents
- 05:02:29 RAG Course (~2 hrs) - traditional + agentic RAG
- 07:10:43 Vectorless RAG (~50 min) - retrieval without vector DB
- 08:02:11 Deep Agents (~43 min) - multi-step research agents
- 08:45:43 Guardrails (~37 min) - safety and validation
- 09:22:55 LLM Evaluation (~1 hr) - testing AI quality
- 10:30:25 LLM Gateways (~30 min) - production routing and cost control

PART 1 - VIDEO NOTES (Simple Language)

Section 0: Introduction (00:00:00)

Generative AI = AI that creates new text, code, or images (like ChatGPT). Agentic AI = AI that can PLAN, USE TOOLS, and ACT on its own to finish a task.

Simple analogy:

- GenAI chatbot = a smart person who only talks.
- Agentic AI = a smart assistant who can search the web, run code, send emails, and loop until the job is done.

Example: You ask 'Book me a cheap flight to Mumbai.' A chatbot gives tips. An agent searches flights, compares prices, and could book if allowed.

This course uses Python, UV package manager, virtual environments, and APIs from OpenAI, Google Gemini, and Groq.

Section 1: LangChain Course (00:02:31)

What is LangChain?

LangChain is a toolkit to connect LLMs with tools, memory, prompts, and agents. Think of it as LEGO blocks for AI apps.

Setup (from video)

- Install UV package manager (Windows PowerShell one-liner from docs.astral.sh/uv)
- Create virtual environment: `uv venv`
- Store API keys in .env file (GOOGLE_API_KEY, GROQ_API_KEY, OPENAI_API_KEY)
- Use python-dotenv: `load_dotenv()`

Model Integration - Two Ways

```
# Way 1: Unified init
from langchain.chat_models import init_chat_model
model = init_chat_model('google_genai:gemin-2.0-flash')

# Way 2: Provider class
from langchain_google_genai import ChatGoogleGenerativeAI
model = ChatGoogleGenerativeAI(model='gemin-2.0-flash')
```

Example: `model.invoke('Hello')` returns AI reply. Same pattern works for Groq models.

Streaming and Batch

- `stream()` = answer comes word-by-word (good for chat UI)
- `batch()` = many questions at once (good for bulk processing)

```
for chunk in model.stream('Explain AI'): print(chunk.content, end='')
```

Tools

Tools let the model DO things, not just talk. A tool has a name, description, and Python function.

```
from langchain.tools import tool

@tool
def get_weather(city: str) -> str:
    '''Get weather for a city'''
    return f'Sunny in {city}'

model_with_tools = model.bind_tools([get_weather])
```

3-step tool loop: (1) User asks -> (2) Model picks tool -> (3) Tool runs -> (4) Model gives final answer.

Messages

- `SystemMessage` = instructions ('You are a helpful assistant')
- `HumanMessage` = user input
- `AIMessage` = model reply (may include `tool_calls`)
- `ToolMessage` = result from running a tool

Structured Output

Force the model to reply in a fixed format (JSON-like) using Pydantic models.

```
class Movie(BaseModel):
    title: str
    year: int

structured = model.with_structured_output(Movie)
result = structured.invoke('Details of Inception')
```

create_agent() - LangChain v1

The modern way to build agents. Handles tool loop automatically.

```
from langchain.agents import create_agent

agent = create_agent(
    model='groq:openai/gpt-oss-20b',
    tools=[get_weather],
    system_prompt='You are helpful.'
)
agent.invoke({'messages': [{'role': 'user', 'content': 'Weather in Pune?'}]})
```

Middleware

- SummarizationMiddleware - compress long chat history
- HumanInTheLoopMiddleware - pause for human approval before risky tools
- Model/Tool call limits - prevent infinite loops and runaway cost

Section 2: LangGraph Course (02:35:12)

Why LangGraph?

LangChain `create_agent()` is easy but limited. LangGraph lets YOU design the exact flow using a graph: nodes (steps) + edges (connections). Built for production agents.

Core Concepts

- State = shared memory (usually messages list)
- Node = one function/step (call LLM, run tools, grade docs)
- Edge = connection between nodes
- Conditional Edge = route based on condition (tool call? yes/no)

ReAct Agent Graph (most common pattern)

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.prebuilt import ToolNode, tools_condition

builder = StateGraph(MessagesState)
builder.add_node('agent', call_llm)
builder.add_node('tools', ToolNode(tools)) # name MUST be 'tools'
builder.add_edge(START, 'agent')
builder.add_conditional_edges('agent', tools_condition)
builder.add_edge('tools', 'agent') # loop back
graph = builder.compile()
```

Flow: START -> LLM -> (if tool needed) Tools -> LLM again -> END

Common mistake: naming node 'tool' but edge says 'tools' -> ValueError.

Checkpointing (Memory across turns)

```
from langgraph.checkpoint.memory import InMemorySaver

graph = builder.compile(checkpointer=InMemorySaver())
config = {'configurable': {'thread_id': 'user-123'}}
graph.invoke(input, config)
```

`thread_id` = unique conversation ID. Same ID = agent remembers previous messages.

Human-in-the-Loop

Agent pauses before sending email or deleting data. Human approves, edits, or rejects.

```
from langgraph.types import Command

if '__interrupt__' in result:
    graph.invoke(Command(resume={'decisions': [{'type': 'approve'}]}), config)
```

`create_react_agent()` shortcut

Prebuilt one-liner if you don't need custom nodes:

```
from langgraph.prebuilt import create_react_agent
graph = create_react_agent(llm, tools)
```

Section 3: RAG Course (05:02:29)

What is RAG?

RAG = Retrieval-Augmented Generation. Instead of asking the LLM from memory alone, you first SEARCH your documents, then give those docs to the LLM as context.

Why? LLMs don't know your company data. RAG grounds answers in YOUR files.

Example: Employee asks 'What is our leave policy?' -> search HR PDF -> LLM answers from that PDF.

Traditional RAG Pipeline (5 steps)

- 1. Load documents (PDF, web, CSV)
- 2. Split into chunks (500-1000 chars each)
- 3. Convert chunks to embeddings (vectors)
- 4. Store in vector database (Chroma, FAISS, Pinecone)
- 5. At query time: find similar chunks -> pass to LLM -> generate answer

Key RAG Concepts

- Embeddings = numbers representing text meaning
- Semantic search = find by meaning, not exact keywords
- Chunk size matters: too small = lost context; too large = noisy retrieval
- Top-K = how many chunks to retrieve (usually 3-5)

Agentic RAG (smarter than basic RAG)

Basic RAG always retrieves once. Agentic RAG lets the LLM DECIDE:

- Should I search documents or answer directly?
- Are retrieved docs good enough? If not, rewrite query and search again.
- Grade document relevance before generating answer.

Agentic RAG Graph Nodes

- generate_query_or_respond - LLM decides: search or answer
- retrieve - fetch documents from vector store
- grade_documents - check if docs are relevant
- rewrite_question - improve query if docs are bad
- generate_answer - final answer from good docs

Example flow: User asks about LangGraph -> agent searches docs -> grades them -> if bad, rewrites question -> searches again -> answers.

RAG vs Fine-Tuning vs Prompt Engineering

- Prompt engineering = change how you ask (cheapest, fastest)
- RAG = add external knowledge at query time (best for docs/FAQ)
- Fine-tuning = retrain model on your data (expensive, for style/behavior)

Section 4: Vectorless RAG (07:10:43)

What is Vectorless RAG?

Normal RAG uses vector databases and embeddings. Vectorless RAG finds information WITHOUT vector search - using keyword search (BM25), document trees, or LLM navigation.

Why skip vectors?

- No vector DB to maintain (simpler, cheaper)
- Preserves document structure (sections stay intact)
- Better for exact facts, tables, and structured docs
- Every retrieval step is explainable (not a black-box similarity score)

How it works (PageIndex / Tree approach)

- 1. Build a tree/table-of-contents from document sections
- 2. LLM reads the map (titles + summaries)
- 3. LLM picks which sections to read (reasoning, not similarity)
- 4. Fetch full sections (not broken chunks)

Analogy: Vector RAG = searching by 'vibe'. Vectorless = reading the book index and going to the right chapter.

Methods in vectorless RAG

- BM25 / keyword search (Elasticsearch, whoosh)
- SQL / structured queries
- Document tree navigation (PageIndex, Vectorless engine)
- Hybrid = vector + BM25 combined (best of both)

LangChain implementation

Subclass BaseRetriever - plug any search backend without vector store:

```
class BM25Retriever(BaseRetriever):
    def _get_relevant_documents(self, query):
        # return matching Document objects
        ...

chain = retriever | prompt | llm
```

Section 5: Deep Agents (08:02:11)

What are Deep Agents?

Deep agents = AI systems that do DEEP RESEARCH over many steps. They plan, search multiple sources, reflect, and produce detailed reports - not one quick answer.

Like a research assistant that spends 10 minutes investigating instead of 10 seconds guessing.

Characteristics

- Multi-step planning (break big task into subtasks)
- Multiple tool calls (web search, doc search, calculator)
- Reflection - agent checks its own work and retries if wrong
- Long-running workflows (minutes, not seconds)

Deep Research Agent pattern

- Planner node - creates research plan
- Researcher node - searches web/docs for each subtopic
- Writer node - compiles findings into report
- Critic node - checks facts and requests more research if needed

Example: 'Research AI trends in 2026' -> plan 5 subtopics -> search each -> summarize -> write report -> verify -> deliver.

Multi-Agent Systems

Multiple specialized agents working together:

- Retriever Agent - finds information
- Summarizer Agent - condenses findings
- Fact-Checker Agent - validates claims
- Writer Agent - produces final output

Section 6: Guardrails (08:45:43)

What are Guardrails?

Safety rules that check AI input and output BEFORE and AFTER the LLM runs. Like airport security for AI.

Types of Guardrails

- Input guardrails - block harmful/off-topic user messages
- Output guardrails - filter toxic, wrong, or leaked data in responses
- Tool guardrails - restrict which tools agent can use
- Human-in-the-loop - require approval for risky actions

Common Guardrail Checks

- PII detection - block personal data (SSN, credit card)
- Topic restriction - only answer HR questions, reject others
- Hallucination check - verify answer against retrieved docs
- Jailbreak detection - block prompt injection attacks

Implementation in LangGraph

Add guardrail nodes in the graph BEFORE and AFTER LLM:

```
def input_guard(state):
    if contains_pii(state['messages'][-1]):
        return {'messages': [AIMessage('Cannot process sensitive data')]}
    return state

builder.add_node('input_guard', input_guard)
builder.add_edge(START, 'input_guard')
builder.add_edge('input_guard', 'agent')
```

Tools: NeMo Guardrails, Guardrails AI, custom Pydantic validators, LangChain middleware.

Section 7: LLM Evaluation (09:22:55)

Why evaluate LLMs?

LLM outputs are random and can change. Without testing, you ship bugs to users. Evaluation = measuring if your AI is good enough.

What to measure

- Correctness - is the answer factually right?
- Relevance - does it answer the actual question?
- Faithfulness - is it grounded in retrieved docs (no hallucination)?
- Latency - how fast?
- Cost - tokens used per request

Evaluation Methods

- Human evaluation - experts rate answers (gold standard, slow)
- LLM-as-judge - another LLM scores the answer (fast, scalable)
- Automated metrics - exact match, BLEU, ROUGE (limited for open-ended text)
- RAG-specific: context precision, context recall, answer relevancy

RAG Evaluation Example

```
# Test dataset
test_cases = [
    {'question': 'Leave policy?', 'expected_contains': '15 days'},
    {'question': 'Remote work?', 'expected_contains': 'hybrid'},
]

# Run pipeline, check if answer contains expected text
for case in test_cases:
    answer = rag_pipeline.invoke(case['question'])
    score = case['expected_contains'] in answer
```

Tools: LangSmith (tracing + eval), RAGAS framework, Phoenix/Arize, custom pytest suites.

Agent Evaluation

- Did agent pick the right tool?
- Did it finish within step limit?
- Did it stay within budget?
- Regression testing - run same tests after every code/model change

Section 8: LLM Gateways (10:30:25)

What is an LLM Gateway?

A middle layer between your app and AI providers (OpenAI, Gemini, Groq). One API for all models. Handles routing, cost tracking, rate limits, and fallbacks.

Analogy: LLM Gateway = electricity board. Your app plugs in one socket; gateway manages all power companies.

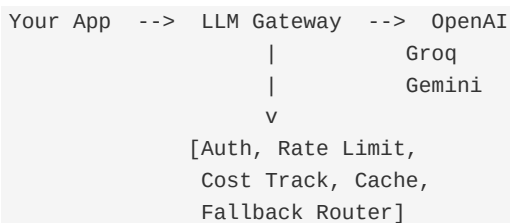
Why you need it in production

- Switch models without changing app code
- Track cost per team/user
- Rate limiting (prevent \$47K runaway agent bills)
- Fallback - if OpenAI fails, auto-switch to Groq
- Caching - return cached answers for repeated questions
- Logging and audit trail

Popular Gateway Tools

- LiteLLM - open source, self-hosted, OpenAI-compatible API
- Portkey - managed gateway with budgets, observability, governance
- LangSmith - tracing and monitoring (LangChain ecosystem)

Gateway Architecture



Example: Team A gets cheap model (Groq). Team B gets premium (GPT-4). Gateway enforces \$100/month budget per team.

PART 2 - TOP INTERVIEW QUESTIONS

Compiled from LinkedIn posts (Rashmi Roy 2026), InterviewBaba, CloudSoftSol, TowardsAI, Glassdoor-style fresher panels, and production-focused system design guides. Grouped by course topic. Practice explaining with real examples.

LangChain & Basics

Q: What is LangChain?

A: Framework to build LLM apps by connecting models, prompts, tools, memory, and agents.

Like React for frontend - LangChain gives reusable pieces for AI apps.

Q: Difference between Chain, Agent, and Tool?

A: Chain = fixed steps. Tool = one action function. Agent = LLM decides which tools to use in a loop.

Chain: summarize doc. Agent: research topic using web+calc tools.

Q: What is create_agent() vs AgentExecutor?

A: create_agent() is LangChain v1 standard, built on LangGraph. AgentExecutor is legacy.

Use create_agent() for new projects.

Q: What are LangChain Messages?

A: SystemMessage, HumanMessage, AIMessage, ToolMessage - building blocks of conversation state.

ToolMessage links back to tool_call_id from AIMessage.

Q: What is structured output?

A: Forcing LLM to return data matching a schema (Pydantic/TypedDict).

Movie(title='Inception', year=2010) instead of free text.

Q: Stream vs Batch?

A: Stream = token-by-token for UX. Batch = many prompts in one run for throughput.

Chat UI uses stream. Bulk tagging uses batch.

LangGraph & Agentic AI

Q: What is Agentic AI?

A: AI that plans, uses tools, and acts autonomously - not just generates text.

Agent searches web + calculates + writes report.

Q: What is ReAct?

A: Reason + Act loop: Thought -> Action (tool) -> Observation -> repeat.

Thought: need weather. Action: get_weather('Pune'). Observation: sunny.

Q: Why LangGraph when LangChain exists?

A: LangGraph gives full control: custom nodes, branching, loops, checkpointing, HITL.

Escalate to human when confidence < 70%.

Q: What are Nodes and Edges?

A: Node = step/function. Edge = flow between steps. Conditional edge = if/else routing.

agent -> tools_condition -> tools OR END.

Q: What is State in LangGraph?

A: Shared data passed between nodes, usually message history.

MessagesState = {'messages': [...]}

Q: What is checkpointing?

A: Saving agent state so conversation continues across turns or after crashes.

InMemorySaver + thread_id='user-42'.

Q: How does agent know when to stop?

A: Final answer, max iterations, budget limit, or error circuit breaker.

Missing step cap caused \$47K runaway agent incident (2025).

Q: ReAct vs Plan-and-Execute?

A: ReAct = one step at a time, adapts. Plan-and-Execute = plan all steps first, then execute.

ReAct for exploration. Plan-and-Execute for known workflows.

Q: What is MCP?

A: Model Context Protocol - standard way for agents to connect to external tools/data.

Like USB port for AI tools.

RAG

Q: What problem does RAG solve?

A: LLMs lack private/current data. RAG adds your documents at query time.

Company FAQ from internal PDFs.

Q: Semantic vs keyword search?

A: Semantic = meaning similarity (embeddings). Keyword = exact word match (BM25).

'car' matches 'automobile' in semantic search.

Q: Why does chunk size matter?

A: Too small = lost context. Too large = irrelevant noise in retrieval.

500-1000 tokens is common starting point.

Q: RAG vs Fine-tuning vs Prompting?

A: Prompting = cheapest. RAG = external knowledge. Fine-tuning = change model behavior.

Use RAG for company docs, not fine-tuning.

Q: What is Agentic RAG?

A: Agent decides when to retrieve, grades docs, rewrites query, retries.

Bad docs -> rewrite question -> search again.

Q: How evaluate RAG?

A: Context precision/recall, answer relevancy, faithfulness, latency.

LangSmith evals + RAGAS metrics.

Q: What is Vectorless RAG?

A: Retrieval without vector DB - BM25, doc trees, LLM navigation.

PageIndex: LLM reads table of contents, picks sections.

Q: Hybrid retrieval?

A: Combine vector + keyword search (Reciprocal Rank Fusion).

Best accuracy in production RAG systems.

Guardrails, Eval, Production

Q: What are LLM Guardrails?

A: Safety checks on input/output/tools to prevent harm, leaks, and bad answers.

Block PII, require HITL for email sending.

Q: How prevent hallucinations in Agentic RAG?

A: Grade retrieved docs, require citations, fallback to 'I don't know', human review.

Answer only if doc relevance score > threshold.

Q: How test non-deterministic agents?

A: Golden test sets, LLM-as-judge, trace inspection, regression suites, budget caps in tests.

Same 50 questions after every deploy.

Q: What is an LLM Gateway?

A: Central API layer for routing, cost control, rate limits, fallbacks across providers.

LiteLLM/Portkey between app and OpenAI/Groq.

Q: Design production RAG for 10M docs?

A: Distributed indexing, hybrid search, reranking, caching, eval pipeline, monitoring.

Shard vector DB, async ingestion, semantic cache.

Q: How monitor cost/latency/quality?

A: LangSmith traces, token counting, dashboards, alerts on budget/latency spikes.

Alert if daily spend > \$50.

Q: How debug looping agent at 3am?

A: Check traces, step count, duplicate tool calls, context window overflow, missing stop condition.

LangSmith shows same tool called 50 times.

PART 3 - QUICK REVISION CHEATSHEET

- LangChain = build blocks | LangGraph = custom agent flow | RAG = docs + LLM
- Tool = function LLM can call | Agent = LLM + tools + loop
- Messages: System, Human, AI, Tool
- create_agent() = easy agent | StateGraph = custom agent
- RAG: Load -> Chunk -> Embed -> Store -> Retrieve -> Generate
- Agentic RAG: retrieve + grade + rewrite + retry
- Vectorless RAG: no vector DB, use BM25 or doc trees
- Deep Agents: multi-step research with planning + reflection
- Guardrails: safety before/after LLM and tools
- Eval: correctness, faithfulness, latency, cost
- Gateway: one API, many models, cost control, fallbacks
- Production must-haves: step limits, budget caps, tracing, evals, HITL for risky tools

Recommended Study Order (Beginner)

- 1. LangChain basics (models, tools, messages) - your notebooks 1-4
- 2. Structured output - notebook 5
- 3. Middleware + checkpointing - notebook 6
- 4. LangGraph StateGraph + ToolNode - your recent code
- 5. Traditional RAG (vector store)
- 6. Agentic RAG (grading + rewrite loop)
- 7. Guardrails + Evaluation + Gateway for production

Sources: YouTube course by Krishna (rV3HJ4LEZ7k), LangChain/LangGraph docs, InterviewBaba Agentic AI 2026, LinkedIn GenAI interview posts, CloudSoftSol, TowardsAI 40 GenAI Questions, LLM Gateway system design guides.